

Attack Detection in Logs

Controlled Social Engineering Simulation — Detection, Containment, Eradication & Post-Incident Review

Assessment Scope

This report documents a controlled, local security exercise. The observed credential records and network activity are treated as test data. The exercise did not target external networks, live accounts, or production systems.

1. Executive Summary

The exercise demonstrated how a locally hosted phishing-style authentication page can generate observable evidence in application and connection logs. The key findings were local capture of test credentials, identification of the client session through loopback networking, and the presence of a PHP-based web service on TCP port 8080.

The incident was contained by terminating the test framework and its web server, removing the extracted test artifacts, invalidating the associated test session/credentials, and verifying that no process remained bound to port 8080. Because the activity was restricted to the local host, the assessed impact was limited to the test environment.

2. Attack Detection in Logs

2.1 Unauthorized Credential Harvesting Activity

Observation: Inspection of the server/application logs showed that information entered into the simulated login page was intercepted and written to local files.

Security significance: This is a high-value detection signal because a normal authentication service should not silently write users' plaintext passwords into arbitrary local data files. In a real environment, such behavior could indicate credential theft, a compromised application, malicious debugging code, or an unauthorized phishing service.

Recommended log evidence: Preserve timestamps, process names/PIDs, file creation or modification events, source/destination addresses, listening ports, and relevant web-server access records before deleting artifacts, subject to the organization's evidence-retention policy.

2.2 Exfiltrated Credentials / Test Credentials

The test artifact auth/usernames.dat contained the following simulated credential pair:

```
Username: absbabsa  
Password: saaaaaaaaa
```

Security significance: Plaintext password storage creates immediate confidentiality risk. Even when the credentials are only test credentials, the same logging pattern in a production system could expose accounts to unauthorized access. Passwords should not be written to ordinary application logs or plaintext files.

Important handling note: The values above are reproduced only because they were supplied as part of a controlled lab report. In a real incident, credentials should be redacted from reports, tickets, screenshots, and shared evidence wherever possible.

2.3 Origin & Session Identification

The file auth/ip.txt recorded the client endpoint as 127.0.0.1 connecting to port 8080.

Interpretation: 127.0.0.1 is the IPv4 loopback address. It indicates that the observed client and server communication occurred on the same host rather than proving that an external machine connected to the service. The port number identifies the local service endpoint used by the test web server.

Detection value: Correlating a loopback connection with a newly started web process and unusual authentication-page files can help analysts distinguish a local security exercise or malicious local service from ordinary remote web traffic.

3. Attack Detection Workflow

Stage	Evidence to Check	Expected Security Interpretation
1. Identify suspicious input handling	Application/server logs and file writes	Credentials or form input are being stored unexpectedly.
2. Identify captured artifacts	auth/usernames.dat and related files	Potential plaintext credential exposure.
3. Correlate network activity	auth/ip.txt, ss/netstat, web-server logs	Map the client endpoint to the local service.
4. Identify responsible process	Process list, listening sockets, parent/child processes	Determine which framework or server created the service.
5. Contain	Terminate unauthorized/test processes	Stop further credential capture and access.
6. Eradicate	Remove artifacts and persistence	Prevent reuse or further disclosure of test data.
7. Validate	Repeat socket/process inspection	Confirm the service is no longer listening.

4. Containment & Eradication

Process Termination: The Zphisher framework process and the associated PHP server listening on port 8080 were terminated. Stopping the service prevents additional form submissions from being captured and stops the local phishing simulation from continuing.

Artifact Removal: The extracted test files `auth/ip.txt` and `auth/usernames.dat` were deleted after the relevant evidence had been reviewed. This reduces the chance of accidental disclosure or reuse of captured test credentials. In a formal investigation, evidence should normally be preserved according to chain-of-custody requirements before deletion.

Session Invalidation: The account associated with the test username was treated as compromised for the purpose of the exercise. The corresponding password was reset and active sessions were invalidated. This is the correct defensive response whenever credentials may have been exposed.

Network Binding Isolation: A socket inspection using tools such as `ss` or `netstat` was used to confirm that no leftover process remained bound to local TCP port 8080. This verification is important because terminating a parent process does not always guarantee that every child process or listener has stopped.

5. Verification Commands

For a defensive lab, the following commands can be used to verify whether a service remains bound to TCP port 8080:

```
ss -ltnp | grep ':8080'
netstat -tulnp | grep ':8080'
```

If no output is returned, there is no matching listening socket visible to the current user. Where permissions are required, the command may need to be run with appropriate administrative privileges. The goal is verification of containment, not exploitation.

6. Post-Incident Report

6.1 Incident Summary

A controlled social-engineering simulation was executed using Zphisher v2.3.5. The test framework hosted a cloned Instagram-style authentication portal on `http://127.0.0.1:8080`. User-submitted test data was captured locally, creating artifacts that could then be inspected for incident-detection purposes.

6.2 Root Cause Analysis

The immediate cause was user interaction with a locally hosted, unencrypted HTTP authentication page. Because the service was not protected by HTTPS and was not a legitimate authentication origin, the simulation could capture submitted test values. More broadly, the scenario demonstrates why users should verify the domain, transport security, and legitimacy of authentication pages before entering credentials.

6.3 Impact Assessment

Impact was limited to local testing artifacts. Based on the supplied scenario, no external networks, production systems, or real user credentials were compromised. The captured values should nevertheless be treated as exposed test credentials and should not be reused.

7. Action Items & Preventive Measures

Deploy FIDO2 / WebAuthn MFA: Use phishing-resistant, origin-bound authentication such as FIDO2/WebAuthn. These mechanisms bind authentication to the legitimate website origin, making

stolen passwords substantially less useful to an attacker.

EDR / NDR Detection: Create detections for unauthorized web servers, suspicious PHP processes, unusual login templates, and authentication services running on non-standard ports such as 8080. Correlate process creation with socket-listening events and unexpected file writes.

Phishing Awareness Training: Train users to inspect the full URL, verify the expected domain, recognize HTTP or certificate warnings, and avoid entering passwords into suspicious or unexpected login pages.

Credential Hygiene: Never store passwords in plaintext logs. Use appropriate password hashing for password storage, minimize sensitive logging, restrict file permissions, and redact secrets from incident reports.

Least Privilege & Application Control: Restrict who can start local web services and frameworks in managed environments. Use application-control policies and endpoint monitoring where appropriate.

Continuous Monitoring: Centralize security logs and alert on anomalous authentication pages, suspicious local listeners, unexpected credential-file creation, and process/network combinations associated with unauthorized services.

8. Detection Indicators Summary

Indicator	Meaning	Recommended Response
auth/username.dat	Unexpected plaintext credential artifact	Secure/redact evidence; reset affected test/real credentials as applicable.
auth/ip.txt	Client/session endpoint record	Correlate with process and web-server logs.
127.0.0.1:8080	Local loopback service endpoint	Identify listening process and confirm whether it is authorized.
Unexpected PHP web server	Potential unauthorized web service	Identify owner/process tree; terminate if unauthorized.
Cloned login page	Potential phishing simulation or malicious portal	Validate authorization; isolate unauthorized instances.

9. Conclusion

The exercise provides a practical example of incident detection through correlation of file artifacts, authentication records, process information, and network sockets. The most important lesson is that credential-capture behavior can often be detected by looking beyond a single log source: file-write events, process execution, listening ports, and authentication activity should be correlated. After containment, the environment should be rechecked to confirm that the malicious or unauthorized service is no longer running and that exposed credentials cannot be reused.

Prepared as a defensive cybersecurity lab report. All activity described is intended for authorized, controlled testing environments.